OPEN JOBS GHANA — ENGINEERING PROMPT BLUEPRINT

System Design Specification Prompt

Target: LLM Code Generation / Senior Architecture Reference Engine

> ***Prompt Mode Instruction:*** *Feed the entirety of the text block below into an advanced code generation agent or development environment to produce functional boilerplate, database schemas, and clean architectural route interfaces.*

1. ARCHITECTURAL PHILOSOPHY & CORE CONSTRAINTS

- **Decoupled Billing & Entitlements:** Payments *never* activate advertisements directly. Payments modify User Subscriptions/Entitlements. Subscriptions dictate platform capabilities and the operational rules applied to individual advertisements.
- **Gateway Agnostic Strategy (Strategy Pattern):** The core billing engine must be highly abstract. While **Paystack** is the initial payment provider (supporting MTN Mobile Money, Telecel Cash, Visa, Mastercard), switching to Flutterwave or another gateway must require only changing an adapter module, without modifying any core business or subscription state logic.
- **Zero-Trust Frontend Model:** The frontend layer is treated as entirely untrusted. No state mutations, duration assignments, or subscription activations can occur via client-side instruction. All operations require deterministic, server-side validation and backend-to-backend gateway verification.
- **B2B Isolation:** The system must strictly process platform-centric subscriptions, ad visibility enhancements, and business accounts. It **must not** include freelancer escrow accounts, client-to-freelancer payments, or user-to-user marketplace transaction processing.
- **Rule-Driven Dynamic Configuration:** To prevent hardcoded limitations, all plans, pricing models, listing expiration spans, active limits, and search ranking boosts must be structurally derived from dynamic configuration rules (stored in DB/cache), allowing an administrative operator to alter constraints without editing code.

2. DYNAMIC DATABASE SCHEMA (RELATIONAL BLUEPRINT)

Generate optimized SQL DDL (PostgreSQL compatible) reflecting this schema with appropriate primary keys, foreign keys, indexes for performance, and constraints:

A. Core Configurations (Rule-Driven Foundation)

- `subscription_plans`: `id` (UUID), `name` (e.g., Free, Monthly, Yearly), `price_ghs` (Numeric), `billing_interval_days` (Int), `is_active` (Bool), `metadata` (JSONB - for storing arbitrary configuration flags).
- `plan_entitlements`: `id` (UUID), `plan_id` (FK), `feature_key` (String, e.g., `max_active_ads`, `ad_duration_days`, `search_rank_tier`, `has_premium_badge`), `feature_value` (String/JSONB).

B. Transactional & User State

- `users`: `id` (UUID), `name`, `email` (Unique), `phone`, `account_type` (Enum: Individual, SME, Corporate, NGO, Gov), `created_at`.
- `subscriptions`: `id` (UUID), `user_id` (FK), `plan_id` (FK), `status` (Enum: Active, Expired, Past_Due, Cancelled), `starts_at`, `expires_at`, `auto_renew` (Bool), `updated_at`.
- `payments`: `id` (UUID), `user_id` (FK), `subscription_id` (FK, Nullable), `amount_ghs`, `reference` (Unique, indexable), `provider` (Enum: Paystack, Flutterwave), `method` (Enum: MoMo, Card, Transfer), `status` (Enum: Success, Pending, Failed, Cancelled, Abandoned), `gateway_response` (JSONB), `paid_at`, `created_at`.

C. Listing & Analytics Engine

- `ads`: `id` (UUID), `user_id` (FK), `title`, `category`, `subcategory`, `status` (Enum: Draft, Pending_Review, Active, Expired, Rejected, Deleted), `created_at`, `activated_at`, `expires_at`.
- `ad_analytics_snapshots`: `id` (UUID), `ad_id` (FK), `metric_key` (Enum: views, messages, bookmarks, phone_clicks, email_clicks, search_appearances), `total_count` (Int), `updated_at`.

3. STRICT ALGORITHMIC & WORKFLOW ENGINE DESIGN

Workflow A: The Two-Step Secure Payment Cycle

1. Initiation: When a user checks out, generate a deterministic, secure reference formatted as: `OJG_[YYYYMMDD]_[CryptographicAlphaNumericString]`. Write a `Pending` state record to the `payments` table before rendering the gateway payment view.

2. Backend-to-Backend Verification: Implement a strict verification workflow handler that consumes both incoming Paystack Webhooks and a fallback background synchronization cron job. Protect the webhook endpoint from replay attacks and enforce payload signature validation using a shared cryptographic secret. On confirmation via Paystack API matching `status == 'success'`, invoke a transactional unit of work that updates the `payment` to `Success`, upserts/renews the record in `subscriptions`, and updates the corresponding entitlement tokens.

Workflow B: The Idempotent Ad Expiration & Reactivation Cron Engine

1. Daily Expiration Sweeper: Design a cron job query that runs daily. Any ad where `NOW() >= expires_at` and `status == 'Active'` must transit to `Expired`. It *must not* be deleted. It must be cleanly filtered out from public-facing search query APIs while remaining completely accessible inside the user's private dashboard under an "Expired" filter. All historical analytics points must be preserved.

2. Reactivation Evaluation Engine: When a user hits the `Reactivate` endpoint for an expired ad, evaluate the user's current active subscription rules: Query the user's active configuration limits (`max_active_ads`). Check if the count of their current `Active` ads violates the entitlement limit. If yes, throw a `PlanLimitExceeded` error code. If valid, update the ad state to `Active`, set `activated_at = NOW()`, and calculate `expires_at = NOW() + INTERVAL 'X days'` where **X** is sourced dynamically from the user's active `plan_entitlements` configurations (e.g., 7 days for Free, 30 for Monthly, 365 for Yearly).

Workflow C: Fraud & Duplicate Ad Interceptor Engine

Before an ad can transition into `Pending_Review` or `Active`, pipe the payload through a strict duplication evaluation layer. Compare the `title`, `category`, `subcategory`, and descriptions against existing records owned by the same `user_id`. If a string metric threshold (e.g., Levenshtein distance or a cosine similarity vector match) indicates a duplication match, block processing and return a structured warning payload: `{"error": "DUPLICATE_AD_DETECTED", "conflicting_ad_id": "UUID", "action_required": ["Reactivate_Existing", "Edit_Existing", "Override_Create"]}`.

Workflow D: Dynamic Search Ranking Engine

Public search query builders must compute listing order dynamically based on account tier entitlements rather than static database fields. Establish a multi-tiered sorting architecture:

> ***Rank Score Group = [1: Verified Premium] → [2: Verified Free] → [3: Standard Premium] → [4: Standard Free]***

Within each identical classification level, rows **must** be sorted secondarily by `activated_at DESC` (most recent activation timestamp), ensuring that active engagement and deliberate reactivations are rewarded over duplicate spam posting patterns.

4. EXPECTED DELIVERABLES

When implementing this prompt, please provide:

1. **Clean Object-Oriented/Functional System Components** implementing the business logic for subscription state updates, payment gateway abstraction interfaces, and the ad reactivation flow.
2. **The Database DDL Schema or ORM definitions** matching Section 2 precisely, including indexing structures optimized for the search queries and payment references.
3. **An API Design Blueprint** mapping out the critical endpoints (`POST /api/v1/payments/initialize`, `POST /api/v1/payments/webhook`, `POST /api/v1/ads/{id}/reactivate`) with input/output structural validation contracts. Ensure proper handling of failure states (`Pending`, `Failed`, `Cancelled`, `Abandoned`).